UNIT-III

Objective:

To understand context free grammars.

Syllabus:

Formal Languages, Context Free Grammar, Leftmost and Rightmost Derivations, Parse Trees, Ambiguous Grammars, Simplification of Context Free Grammars-Elimination of Useless Symbols, Є-Productions and Unit Productions, Normal Forms-Chomsky Normal Form and Greibach Normal Form, Pumping Lemma, Closure Properties, Applications of Context Free Grammars.

Learning Outcomes:

Students will be able to:

- construct Context free grammar for the given language.
- construct right most, left most derivation and derivation trees for the given string and grammar.
- optimize the productions by simplification methods like useless symbols elimination, unit production and Є-productions.
- verify the grammar is CGF or not using pumping lemma theorem.

Learning Material

3.1 Formal Languages

Definition: A formal language is a set of strings, where each string consists of symbols from a specific alphabet. These strings are well-formed according to a predefined set of rules.

Examples:

- The set of all binary strings (strings made of 0s and 1s) is a formal language.
- The set of all syntactically correct Java programs is another example.

3. 2 Context-Free Grammar:

A context-free grammar (CFG or just grammar) is denoted G = (V, T, P, S), where

- V and T are finite sets of variables and terminals, respectively.
- P is a finite set of productions; each production is of the form A → α, where A is a variable and α is a string of symbols from (V ∪ T)*.
- S is a special variable called the start symbol.

The language generated by G [denoted L(G)] is {w | w is in T* and $S \overset{*}{\underset{G}{\Rightarrow}} w$}. That is, a string is in L(G) if:

1) The string consists solely of terminals.
2) The string can be derived from S.

We call L a context-free language (CFL) if it is L(G) for some CFG G.

Note: **C language is an example for Context Free Language.**

Examples:

1. Write CFG for the language L= $\{a^n b^n \mid n>=1\}$.

   L= {ab, aabb, aaabbb, aaaabbbb, aaaaabbbbb, ……}

   G = ({S}, {a, b}, P, S)

   P: S -> aSb | ab

   (Or)

   S -> aSB

   S -> aB

   B -> b

2. Write CFG for the language L= $\{a^n b^m \mid n , m >=1\}$.

   L= {a, b, ab, aab, abb,aabb, aaabbb, aaaabbbb, aaaaabbbbb,……}

   G = ({S, A, B}, {a, b}, P, S)

   P: S -> AB

   A -> aA | a

   B -> bB | b

3. Write CFG for the language L={aa,ab,ba,bb}

   G = ({S, A}, {a, b}, P, S)

   P: S -> AA

   A -> a | b

4. Write CFG for the language L= { a^n | n>=0}.

   L= { ε, a, aa, aaa, aaaa, aaaaa, aaaaaa,……}

   G = ({A}, {a}, P, A)

   P: A -> aA | ε

5. Write CFG for the regular expression (a+b)*.

   L= { ε, a, b, aa, ab , ba, bb, aaa,abb,aba,……}

   G = ({S}, {a, b}, P, S)

   P: S -> aS | bS | ε

6. Write CFG to generate all strings of {a, b} whose length is atleast 2.

   L= { aa, ab , ba, bb, aaa,abb,aba,……}

   G = ({S, A, B}, {a, b}, P, S)

   P: S -> AAB

   A -> a | b

   B -> aB | bB | ε

7. Write CFG to generate all strings of {a, b} whose length is atmost 2.

   L= { ε , a,b, aa, ab , ba, bb}

   G = ({S, A}, {a, b}, P, S)

   P: S -> AA

   A -> a | b | ε

8. Write CFG to generate palindromes over {a, b}.

   L= { ε , a,b, aa,bb,aba,bab,aaaa,abba,……}

   G = ({S}, {a, b}, P, S)

   P: S -> aSa | bSb

   S -> a | b | ε

9. Write CFG for the language L= { $a^n b^{2n}$ | n>=1}.

   L= { abb, aabbbb, aaabbbbbb, aaaabbbbbbbb,……}

   G = ({S}, {a,b }, P, S)

   P: S-> aSbb | abb

10. Write CFG for the language L= { $a^n b^n c^m$ | n, m >=1}.

    L= { abc, aabbc,aabbcc, aaabbbc, aaaabbbbcc,……}

    G = ({S,A,B}, {a,b }, P, S)

    P: S-> AB

    A -> aAb | ab

    B -> cB | c

11. Write CFG to generate equal number of a's and b's.

L= { ab, ba,aabb, abab, bbaa,baba,……}

G = (V, T, P, S), where V = {S, A, B}, T = {a, b},S and P .

P: S -> aB A -> bAA

S -> bA B ->b

A ->a B ->bS

A ->aS B -> aBB

3.3 Leftmost and Rightmost Derivations:

Sentential Form:

A string of terminals and variables α is called a sentential form if $S \overset{*}{\Rightarrow} \alpha$.

Derivation:

Derivation is the process of applying productions repeatedly to expand non-terminals in terms of terminals or non-terminals, until there are no more non-terminals.

A derivation can be either Leftmost derivation or Right most derivation.

Leftmost derivation:

If at each step in a derivation a production is applied to the leftmost variable, then the derivation is said to be leftmost.

Example:

Consider the grammar G = ({S, A}, {a, b}, P, S), where P consists of

S →aAS | a

A→ SbA|SS|ba

The corresponding leftmost derivation is

S => aAS => aSbAS => aabAS => aabbaS => aabbaa. **Sentential form**

Rightmost derivation:

A derivation in which the rightmost variable is replaced at each step is said to be rightmost.

Example:

Consider the grammar G = ({S, A}, {a, b}, P, S), where P consists of

S →aAS | a

A→ SbA|SS|ba

The corresponding rightmost derivation is

S => aAS => aAa => aSbAa => aSbbaa => aabbaa.

Note:"If w is in L(G) for CFG G, then w has at least one parse tree, and corresponding to a particular parse tree, w has a unique leftmost and a unique rightmost derivation."

3.4 Parse trees (or) Derivation Trees :

The derivations in a CFG can be represented using trees. Such trees representing derivations are called derivation trees.

Let **G = (V, T, P, S)** be a CFG. A tree is a derivation (or parse) tree for G if:

1) Every vertex has a label, which is a symbol of V ∪ T ∪ {ε}.

2) The label of the root is S.

3) If a vertex is interior and has label A, then A must be in V.

4) If n has label A and vertices n1, n2, n3, ..., nk are the sons of vertex n, in order from the left, with labels X1, X2, ..., Xk, respectively, then

A→X1X2 .....Xk

must be a production in P.

5) If vertex n has label ε, then n is a leaf and is the only son of its father.

Example:

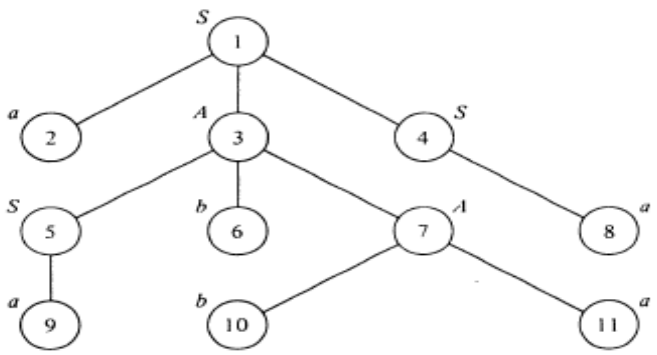
Consider the grammar G = ({S, A}, {a, b], P, S), where P consists of

S →aAS | a

A→ SbA|SS|ba

Construct a derivation tree for the string "aabbaa"

A derivation tree is a natural description of the derivation of a particular sentential form of the grammar G. If we read the labels of the leaves from left to right, we have a sentential form. We call this string the yield of the derivation tree.



S => aAS => aSbAS => aabAS => aabbaS => aabbaa.

Note: Some leaves could be labelled by ε.

3.5 Ambiguous Grammars:

A context-free grammar G is said to be ambiguous if it has two parse trees for some word.

(or)

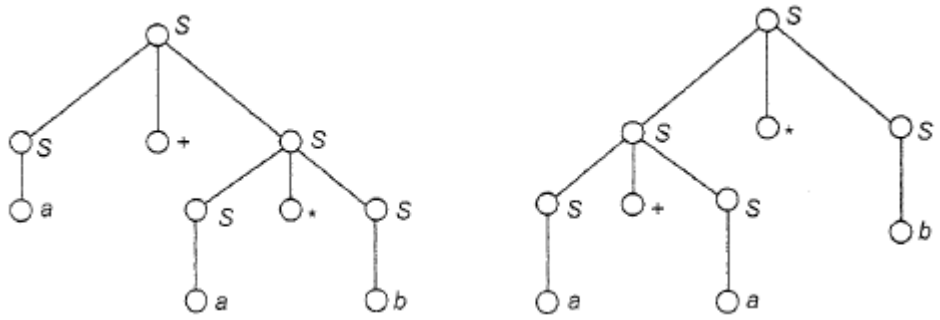
A word which has more than one leftmost derivation or more than one rightmost derivation is said to be ambiguous.

Note: A CFL for which every CFG is ambiguous is said to be an inherently ambiguous CFL.

Example:

G = *({S}, {a, b, +, *}, P. S),* where *P* consists of S→S+S | S*S | a | b

We have two derivation trees for *a* + *a* * *b*



Two derivation trees for a + a * *b*

3.6 Simplification of Context Free Grammars:

- **A Context-Free Grammar** (CFG) is a formal grammar that consists of a set of production rules used to generate strings in a language. However, many grammars contain redundant rules, unreachable symbols, or unnecessary complexities.
- Simplifying a CFG helps in reducing its size while preserving the generated language, making parsing more efficient.

Why Simplify a CFG?

Simplifying a CFG is essential for:

- **Efficient Parsing:** Smaller grammars require fewer computations.

- **Better Understanding:** Removing redundant parts makes it easier to analyze.
- **Optimization:** Improved computational efficiency in compilers and language processing.

Types of redundant productions and the procedure of removing them are mentioned below:

1) Elimination of useless symbols.

2) Elimination of ε –Productions.

3) Elimination of Unit Productions.

Elimination of Useless Symbols:

Let G=(V, T, P, S) be a grammar. A symbol X is useless if it is not involved in derivation.

(or)

A symbol X is useless if there is no way of getting a terminal string from it.

Example:

Consider the grammar

S→AB | a

A→ a

We find that no terminal string is derivable from B. We therefore eliminate B and the production S → AB.

Then the grammar is

S→a

A→a

We find that only S and a appear in sentential forms. Thus ({S}, {a}, {S → a}, S) is an

equivalent grammar with no useless symbols.

Elimination of ε –Productions:

A production of the form $A \rightarrow \varepsilon$, where A is a variable, is called a *null production.*

If L = L(G) for some CFG G = (V, T, P, S), then L - { ε } is L(G') for a CFG G' with no useless symbols or ε -productions.

Example:

Consider the grammar

A→0B1 | 1B1

B→0B | 1B | ε

Remove ε-productions from the grammar.

B→ ε is the null production.

The new productions after elimination of ε are

A→0B1 | 1B1| 01 | 11

B→0B | 1B | 0 | 1

Elimination of Unit Productions:

A production of the form A→B whose right-hand side consists of a single variable is called a unit production.

All other productions, including those of the form A →a and ε -productions, are nonunit productions.

Example:

Consider the grammar

S→0A | 1B | C

A→0S | 00

B→1 | A

C→01

Remove unit production from the grammar.

S→C and B→A are the unit productions

The new productions after elimination of unit productions are

S→0A | 1B | 01

A→0S | 00

B→1 | 0S | 00

C→01

C is a useless symbol. So eliminate C production.

The final set of productions are

S→0A | 1B | 01

A→0S | 00

B→1 | 0S | 00

3.7 Normal Forms:

Context-free grammars (CFGs) have specific normal forms that simplify their structure while preserving the language they generate. The most common normal forms are Chomsky Normal Form (CNF) and Greibach Normal Form (GNF).

Chomsky Normal Form :(CNF)

Any context-free language without ε is generated by a grammar in which all productions are of the form A→BC or A→ a. Here, A, B, and C, are variables and a is a terminal.

Step 1: Simplify the grammar.

a) Eliminate ε –productions

b) Eliminate unit productions

c) Eliminate Useless symbols.

The given grammar does not contain ε –productions, unit productions and useless symbols.

It is in optimized form.

Step 2: Consider a production in P,of the form A->$X_1X_2X_3$.....X_m where m>=2.If X_i is a terminal a, introduce a new variable C_a and a production C_a->a.Then replace X_i by Ca.

Step 3: Consider a production A->$B_1B_2B_3$.....B_m where m>=3,create new variables D_1,D_2,....D_{m-2} and replace A->$B_1B_2B_3$...B_m by the set of productions {A->B_1D_1,D_1->B_2D_2,.....Dm-3->B_{m-2} D_{m-2},Dm-2->$B_{m-1}B_m$ }

Example:

Consider the grammar ({S, A, B}, {a, b}, P, S) that has the productions:

S→ bA | aB

A→bAA | aS | a

B→ aBB | bS | b

Find an equivalent grammar in CNF.

Step 1: Simplify the grammar.

a) Eliminate ε –productions

b) Eliminate unit productions

c) Eliminate Useless symbols.

The given grammar does not contain ε –productions, unit productions and useless symbols.

It is in optimized form.

Step 2: The only productions already in proper form are A→ a and B→b.

So we may begin by replacing terminals on the right by variables, except in the case of the productions A → a and B → b.

S → bA is replaced by S→ C_bA and C_b→ b.

Similarly, A→ aS is replaced by A → C_aS and C_a→ a; A→ bAA is replaced by A→C_bAA; S→ aB is replaced by S→C_aB;

B→bS is replaced by B→ C_bS, and B → aBB is replaced by B→ C_aBB.

In the next stage, the production A→C_bAA is replaced by A → $C_b$$D_1$ and D_1 →AA, and the production B→C_aBB is replaced by B→ $C_a$$D_2$ and D_2 →BB.

Step 3: The productions for the grammar in CNF are :

S→ C_bA | C_aB

A → C_aS | $C_b$$D_1$ | a

B→ C_bS | $C_a$$D_2$ | b

D_1 →AA

D_2 →BB

C_a→ a

C_b→ b

<u>Greibach Normal Form</u>:

Every context-free language L without e can be generated by a grammar for which every production is of the form A → aα, where A is a variable, a is a terminal, and α is a (possibly empty) string of variables.

Lemma 1: Define an A-production to be a production with variable A on the left. Let G = (V, T, P, S) be a CFG. Let A → α1Bα2 be a production in P and B→β1 | β2| ……|βr be the set of all B-productions. Let G1 = (V, T, P1, S) be obtained from G by deleting the production A → α1Bα2 from P and adding the productions A→ α1β1α2 | α1β2α2 | ……| α1βrα2. Then L(G) = L(G1).

Lemma 2: Let G = (V, T, P, S) be a CFG. Let A→Aα1 | Aα2 | ……| Aαr be the set of A-productions for which A is the leftmost symbol of the right-hand side. Let A → β1 | β2| ……|βs be the remaining A-productions. Let G1 = (V U {B}, T, P1, S) be the CFG formed by adding the variable B to V and replacing all the A-productions by the productions:

1) $\left.\begin{matrix} A \to \beta_i \\ A \to \beta_i B \end{matrix}\right\} 1 \le i \le s,$ 2) $\left.\begin{matrix} B \to \alpha_i \\ B \to \alpha_i B \end{matrix}\right\} 1 \le i \le r.$

Then L(G1) = L(G).

Example:

Convert to Greibach normal form the grammar

G=i{A1,A2,A3}, {a, b}, P A1),

where P consists of the following:

$$A_1 \to A_2 A_3$$
$$A_2 \to A_3 A_1 \mid b$$
$$A_3 \to A_1 A_2 \mid a$$

Step 1 Since the right-hand side of the productions for A_1 and A_2 start with terminals or higher-numbered variables, we begin with the production $A_3 \to A_1 A_2$ and substitute the string $A_2 A_3$ for A_1. Note that $A_1 \to A_2 A_3$ is the only production with A_1 on the left.

The resulting set of productions is:

$$A_1 \to A_2 A_3$$
$$A_2 \to A_3 A_1 \mid b$$
$$A_3 \to A_2 A_3 A_2 \mid a$$

Since the right side of the production $A_3 \to A_2 A_3 A_2$ begins with a lower-numbered variable, we substitute for the first occurrence of A_2 both $A_3 A_1$ and b. Thus $A_3 \to A_2 A_3 A_2$ is replaced by $A_3 \to A_3 A_1 A_3 A_2$ and $A_3 \to b A_3 A_2$. The new set is

$$A_1 \to A_2 A_3$$
$$A_2 \to A_3 A_1 \mid b$$
$$A_3 \to A_3 A_1 A_3 A_2 \mid b A_3 A_2 \mid a$$

We now apply Lemma 2 to the productions

$$A_3 \to A_3 A_1 A_3 A_2 \mid b A_3 A_2 \mid a.$$

Symbol B_3 is introduced, and the production $A_3 \to A_3 A_1 A_3 A_2$ is replaced by $A_3 \to b A_3 A_2 B_3$, $A_3 \to a B_3$, $B_3 \to A_1 A_3 A_2$, and $B_3 \to A_1 A_3 A_2 B_3$. The resulting set is

$$A_1 \to A_2 A_3$$
$$A_2 \to A_3 A_1 \mid b$$
$$A_3 \to b A_3 A_2 B_3 \mid a B_3 \mid b A_3 A_2 \mid a$$
$$B_3 \to A_1 A_3 A_2 \mid A_1 A_3 A_2 B_3$$

Step 2 Now all the productions with A_3 on the left have right-hand sides that start with terminals. These are used to replace A_3 in the production $A_2 \to A_3 A_1$ and then the productions with A_2 on the left are used to replace A_2 in the production $A_1 \to A_2 A_3$. The result is the following.

$$A_3 \to bA_3A_2B_3 \qquad A_3 \to bA_3A_2$$
$$A_3 \to aB_3 \qquad A_3 \to a$$
$$A_2 \to bA_3A_2B_3A_1 \qquad A_2 \to bA_3A_2A_1$$
$$A_2 \to aB_3A_1 \qquad A_2 \to aA_1$$
$$A_2 \to b$$
$$A_1 \to bA_3A_2B_3A_1A_3 \qquad A_1 \to bA_3A_2A_1A_3$$
$$A_1 \to aB_3A_1A_3 \qquad A_1 \to aA_1A_3$$
$$A_1 \to bA_3$$
$$B_3 \to A_1A_3A_2 \qquad B_3 \to A_1A_3A_2B_3$$

Step 3 The two B_3-productions are converted to proper form, resulting in 10 more productions. That is, the productions

$$B_3 \to A_1A_3A_2 \quad \text{and} \quad B_3 \to A_1A_3A_2B_3$$

are altered by substituting the right side of each of the five productions with A_1 on the left for the first occurrences of A_1. Thus $B_3 \to A_1A_3A_2$ becomes

$$B_3 \to bA_3A_2B_3A_1A_3A_3A_2, \qquad B_3 \to aB_3A_1A_3A_3A_2.$$
$$B_3 \to bA_3A_3A_2, \qquad B_3 \to bA_3A_2A_1A_3A_3A_2, \qquad B_3 \to aA_1A_3A_3A_2.$$

The other production for B_3 is replaced similarly. The final set of productions is

$$A_3 \to bA_3A_2B_3 \qquad A_3 \to bA_3A_2$$
$$A_3 \to aB_3 \qquad A_3 \to a$$
$$A_2 \to bA_3A_2B_3A_1 \qquad A_2 \to bA_3A_2A_1$$
$$A_2 \to aB_3A_1 \qquad A_2 \to aA_1$$
$$A_2 \to b$$
$$A_1 \to bA_3A_2B_3A_1A_3 \qquad A_1 \to bA_3A_2A_1A_3$$
$$A_1 \to aB_3A_1A_3 \qquad A_1 \to aA_1A_3$$
$$A_1 \to bA_3$$
$$B_3 \to bA_3A_2B_3A_1A_3A_3A_2B_3 \qquad B_3 \to bA_3A_2B_3A_1A_3A_3A_2$$

$$B_3 \to aB_3A_1A_3A_3A_2B_3 \qquad B_3 \to aB_3A_1A_3A_3A_2$$
$$B_3 \to bA_3A_3A_2B_3 \qquad B_3 \to bA_3A_3A_2$$
$$B_3 \to bA_3A_2A_1A_3A_3A_2B_3 \qquad B_3 \to bA_3A_2A_1A_3A_3A_2$$
$$B_3 \to aA_1A_3A_3A_2B_3 \qquad B_3 \to aA_1A_3A_3A_2$$

3.8 Pumping Lemma for CFL's:

Let L be any CFL. Then there is a constant n, depending only on L, such that if z is in L and $|z| \geq n$, then we may write z = uvwxy such that

1) $|vx| \geq 1$,
2) $|vwx| \leq n$, and
3) for all $i \geq 0$ uv^iwx^iy is in L.

Example:

Consider the language $L = \{a^ib^ic^i \mid \geq 1\}$. Suppose L were context free and let n be the constant.

- Consider $z = a^nb^nc^n$. Write $z = uv^iwx^iy$ so as to satisfy the conditions of the pumping lemma.
- Since $|vwx| \leq n$, it is not possible for vx to contain instances of a's and c's, because the rightmost a is n + 1 positions away from the leftmost c.
- If v and x consist of a's only, then uwy (the string uv^iwx^iy with i = 0) has n b's and n c's but fewer than n a's since $|vx| \geq 1$.
- Thus, uwy is not of the form $a^ib^ic^i$. But by the pumping lemma vwy is in L, a contradiction.
- The cases where v and x consist only of b's or only of c's are disposed of similarly.
- If vx has a's and b's, then uwy has more c's than a's or b's, and again it is not in L.
- If vx contains b's and c's, a similar contradiction results.

We conclude that L is not a context-free language.

3.9 Closure Properties:

- Context-free languages are closed under union, concatenation and Kleene closure.
- The context-free languages are closed under substitution.
- The CFL's are closed under homomorphism.
- The CFL's are not closed under intersection.
- The CFL's are not closed under complementation.

3.10 Applications of Context Free Grammars.

1 **Programming Language Design and Implementation**

CFGs are fundamental to defining the syntax of programming languages. The Backus-Naur Form (**BNF**) and Extended Backus-Naur Form (**EBNF**) notations, widely used for specifying programming language grammars. These are essentially context-free grammars. These grammars states the structure of programs, keeping the language's rules.

2 **Compiler Construction: Parsing and Syntax Analysis:**

Compilers translate high-level code into machine-executable instructions. Compilers heavily rely on CFGs during the parsing phase. Parsing involves analyzing the syntactic structure of the input code based on the language's grammar.CFGs are used for the construction of parsers, which are algorithms responsible for −

- **Lexical Analysis** − Grouping characters into meaningful units or tokens.
- **Syntax Analysis** − Verifying if the sequence of tokens conforms to the grammar rules.
- **Building Parse Trees** − Creating tree-like representations of the code's grammatical structure.

3 **Markup Languages and Data Representation –**

Markup languages like HTML and XML, used for web development and data representation, they rely on CFG-based principles to define their structure. These languages use tags to define elements and attributes, and their hierarchical organization can be represented and validated using CFGs.

4 **Natural Language Processing (NLP) –**

Natural language is inherently ambiguous and more complex than formal languages we learn in automata theory. The context-free grammars still play a significant role in NLP as well.

Grammar Formalisms − CFGs are used in computational linguistics to model the grammatical structure of natural language, even if they can't capture all nuances.

Parsing Natural Language − While challenging, parsing natural language with CFGs helps with tasks like identifying parts of speech and understanding sentence structure, these are crucial for applications like machine translation and sentiment analysis.

Assignment-Cum-Tutorial Questions

A. Questions testing the understanding / remembering level of students

I) Objective Questions

1. The C language is [ ]
   a) A context free language b) A context sensitive language
   c) A regular language d) None

2. Every regular grammar is context free grammar. (True | False)

3. The finite automata accepts the following language: [ ]
   a) Context free language b) regular language
   c) Context sensitive language d) all of the above

4. Context-free grammar can be recognized by [ ]
   a) Finite Automata b) Linear bounded Automata
   c) Push down Automata d) both (b) and (c)

5. The language accepted by a Turing Machine: [ ]
   a) Type 0 b) Type 1
   c) Type 2 d) Type 3

6. Match the following

1. Context Free Language	a. Turing Machine	[ ]
2. Recursively Enumerable	b. Finite Automata	[ ]
3. Regular Language	c. Linear Bounded Automata	[ ]
4. Context Sensitive Language	d. Push Down Automata	[ ]

7. A context free grammar is [ ]
   a) Type 0 b) Type 1 c) Type 2 d) Type 3

8. Identify the useless symbol in the grammar given below. [ ]
   S->AB | C A->a B-> BC C->b
   a) S b) A c) B d) C

9. Find an equivalent reduced grammar for the given grammar. [ ]

S-> 0 | 1 | ε S-> 0S0 | 1S1

a) S->0 | 1 ,S->0S0 |1S1 | 0 | 1 b) S->0 | 1 ,S->SS|0S1 | 1S1

c) S-> 0 | 1,S->00 |11 d) None

10. Which one of the following is a Chomsky Normal Form grammar? [ ]

(i) A->BC |a (ii) A->aA |a |b (iii) A->BCD |a, B->a, C->c, D->d

a) (i) only b) (i) and (iii) c) (ii) and (iii) d) (i),(ii) and (iii)

11. L={ 0^n 1^{2n} |n>=1} is [ ]

a) regular b) context-free but not regular c) context-free but regular

d) None

II) Descriptive questions

1. Define Context Free Grammar with an example.
2. What is sentential form? Explain with an example.
3. Explain derivation tree with an example.
4. Define LMD and RMD.
5. Show that the grammar S → 0A | 1B
   A → 0AA | 1S | 1
   B → 1BB | 0S | 0 is ambiguous.
6. Simplify the grammar S→aA|aBB, A→aAA|ε, B→bB|bbc, C→B.
7. Remove all and unit production rules from the following CFG
   S → AaA/ CA/ BaB
   A → aaBa/ CDA / aa / DC
   B → bB / bAB / bb/ aS
   C → Ca / bc / D
   D →bD / A
8. Convert the following grammar to CNF
   S→ aAbB, A→aA|a, B→bB|b.
9. Convert the grammar into GNF S → AA|a, A→ AA|b .

B. Question testing the ability of students in applying the concepts.

I) Multiple Choice Questions:

1. Which word can be generated by S->d| bA ,A->d |ccA [ ]

   a) bccccd b) aabccd c) ababccd d) abbbd

2. Which of the following strings is in the language defined by grammar [ ]

S-> 0A , A-> 1A | 0A | 1

a) 01100 b) 00101 c) 10011 d) 11111

3. Recognize the CFL for the given CFG. [ ]

S-> aB| bA,

A-> a|aS|bAA,

B-> b|bS|aBB

a) strings contain equal number of a's and equal number of b's.

b) strings contain odd number of a's and odd number of b's.

c) strings contain odd number of a's and even number of b's.

d) strings contain even number of a's and even number of b's

4. Given the following productions of a grammar: [ ]

S→aA| aBB; A→aaA | ε ; B→ bB| bbC; C→ B

Which of the following is true ?

A. The language corresponding to the given grammar is a set of even number of a′s.

B. The language corresponding to the given grammar is a set of odd number of a′s.

C. The language corresponding to the given grammar is a set of even number of a′s followed by odd number of b′s.

D. The language corresponding to the given grammar is a set of odd number of a′s followed by even number of b′s.

5. A regular grammar for the language L={$a^n b^m$ | n is even and m is even} is [ ]

a) S→aSb | X; X→bXa | ε

b) S→aaS | X; X→ bSb| ε

c) S→aSb | X; X→ Xab| ε

d) S→aaS | X; X→ bbX | ε

6. Which of the regular expressions corresponds to this grammar? [ ]

S-> AB | AS , A-> a | aA , B->b

a) (aa)*b b) aa*bc) (ab)* d) a(ab)*

10. Identify the language generated by the following grammar [ ]

s→aS | bS | abA

A→aA | bA | ε

a) L= x | ab is a substring of x, x ∈{a,b}*

b) L= x | a is a substring of x, x ∈{a,b}*

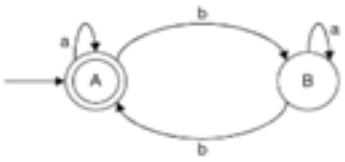
c) L= x | b is a substring of x, x ∈{a,b}*

d) L= x | ba is a substring of x, x ∈{a,b}*

11. The CFG S→aS | bS | a | b is equivalent to the regular expression [ ]

a) (a*+b)* b) (a+b)* c) (a+b) (a+b)* d) (a+b)(a+b)

12. The regular grammar for the given FA is [ ]



a) A→aA | bB | a
B→bA | aB | b

b) A→aA | bB | ε
B→bA | aB | ε

c) A→aA | bB | b
B→bA | aB | a

d) A→bA | aB | a
B→aA | bB | b

13. The number of derivation trees for the string "aabbaa" for the given grammar is

S→aAS | a A→SbA | SS | ba [ ]

a) 4 b) 1 c) 3 d) 2

II) Problems

1. Show that **id+id*id** can be generated by two distinct derivation trees in the grammar

E -> E+E | E*E | (E) | id

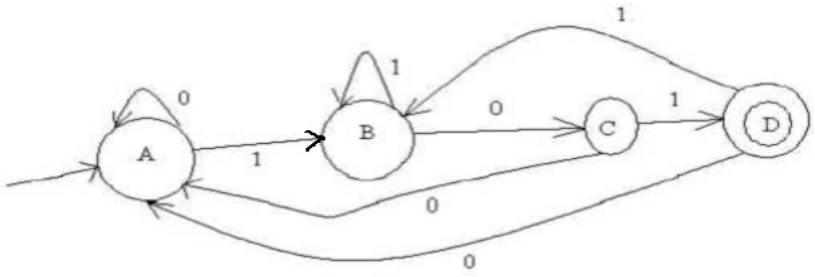
2. Obtain a CFG to generate the set of all strings over alphabet {a, b} with exactly twice as many a's as b's.

3. Obtain a CFG to generate unequal number of a's and b's.

4. Let G = (V, Σ, P, S) be a context-free grammar such that V = {E, T, F}, Σ = {a, +, ∗,(,)}, S = E and P is E → E + T | T T → T * F | F F → (E) | a
   Give parse tree, leftmost and rightmost derivation for the following strings.
   (i) a*(a+a) (ii) ((a+(a)))

5. Give an equivalent left linear grammar for the following right linear grammar
   A0-> aA1, A1->bA1, A1->a , A1-> bA0

6. Obtain a right linear grammar and left linear grammar for the following FA.



7. Construct automata for the following grammar
   S→0S|1A|1
   A→0A|1A|0|1

C. Questions testing the analyzing /evaluating ability of students.

1. State in English about the language corresponding to below given grammar
   S→aB/bA, A→a/aS/bAA , B→b/bS/aBB.

D. GATE/NET/SLET

1. Consider the following context-free grammars: [ ]

GATE 2016 Set-A

*G*1: *S-> aS | B, B->b | bB*
*G*2: *S->aA | bB*; *A-> aA | B | ε , B -> bB | ε*

Which one of the following pairs of languages is generated by $G1$ and $G2$, respectively?

(A) $\{a^m b^n \mid m > 0 \text{ or } n > 0\}$ and $\{a^m b^n \mid m > 0 \text{ and } n > 0\}$

(B) $\{a^m b^n \mid m > 0 \text{ and } n > 0\}$ and $\{a^m b^n \mid m > 0 \text{ or } n >= 0\}$

(C) $\{a^m b^n \mid m >= 0 \text{ or } n > 0\}$ and $\{a^m b^n \mid m > 0 \text{ and } n > 0\}$

(D) $\{a^m b^n \mid m >= 0 \text{ and } n > 0\}$ and $\{a^m b^n \mid m > 0 \text{ or } n > 0\}$

2. S -> aSa | bSb | a | b [ ]

GATE 2009

The language generated by the above grammar over the alphabet {a, b} is the of

a) all palindromes b) all odd length palindromes

c) strings that begin and end with the same symbol d) all even length palindromes

3. Consider the CFG with {S,A,B} as the non-terminal alphabet {a,b} as the terminal alphabet,S as the start symbol and the following set of production rules: [ ]

S -> aB	S -> bA
B -> b	A -> a
B -> bS	A -> aS
B-> aBB	S -> bAA

GATE 2007

Which of the following strings is generated by the grammar?

a) aaaabb b) aabbbb c) aabbab d) abbbba

4. How many derivation trees are there for the grammar in Question 3? [ ]

a) 1 b) 2 c) 3 d) 4

5. . Consider the languages: [ ]

$L_1 = \{ww^R \mid w \text{ belongs } \{0,1\}^*\}$

$L_2 = \{w\#w^R \mid w \text{ belongs } \{0,1\}^*\}$,where # is a special symbol

$L_3 = \{ww \mid w \text{ belongs } \{0,1\}^*\}$

Which one of the following is TRUE? [Gate 2005]

a) L_1 is a deterministic CFL b) L_2 is a deterministic CFL

c) L_3 is a CFL,but not a deterministic CFL d) L_3 is a deterministic CFL

6. . Consider the following statements about the context free grammar [ ]

[GATE-CS-2006]

$$G = \{S \rightarrow SS, S \rightarrow ab, S \rightarrow ba, S \rightarrow E\}$$

I. G is ambiguous

II. G produces all strings with equal number of a's and b's

III. G can be accepted by a deterministic PDA.

Which combination below expresses all the true statements about G?

a) Ionly **b)** I and III only
c) I and II only **d)** I, II and III

8. If L1 is context free language and L2 is a regular language which of the following is/are false? [Gate 1999]
   a) L1-L2 is not context free b) L1 ∩ L2 is context free \
   c) ~L1 d) ~L2
9. Context-free languages are closed under: [ ]
   a) Union, Intersection b) Union, Kleene closure
   c) Intersection, complement d) Complement, Kleene closure [Gate1998]